



# Accessibility Task

[Visit our website](#)

# Introduction

This task introduces essential web accessibility concepts to help you create inclusive, user-friendly websites. You will explore core accessibility principles, focusing on using semantic HTML to improve structure and navigation for screen readers, adding ARIA roles and attributes where HTML alone falls short, and designing accessible forms and inputs that are easy to understand and navigate. By following these practices, you will enhance usability for people with disabilities, ensuring a more accessible web experience for all users.

## Core accessibility principles

Web accessibility is about making websites usable by people with various disabilities. This involves ensuring that all users can perceive, navigate, and interact with your website regardless of their abilities. There are key principles to follow, such as semantic HTML, ARIA roles, accessible forms, and CSS for visual accessibility, that help make your site more inclusive.



### Extra resource

## Web Content Accessibility Guidelines (WCAG)

To put these principles into practice, the **Web Content Accessibility Guidelines** (WCAG) provide a set of rules that help make websites and web content more accessible, especially for people with disabilities. These guidelines cover different areas of design, code, and content to ensure that everyone, including people with visual, hearing, and other disabilities, can use websites easily.

---

## Semantic HTML for accessibility

Semantic HTML is the practice of using HTML tags that convey the meaning of the content they contain. It improves the structure of your web page, making it easier for screen readers and other assistive technologies to understand and navigate your content.

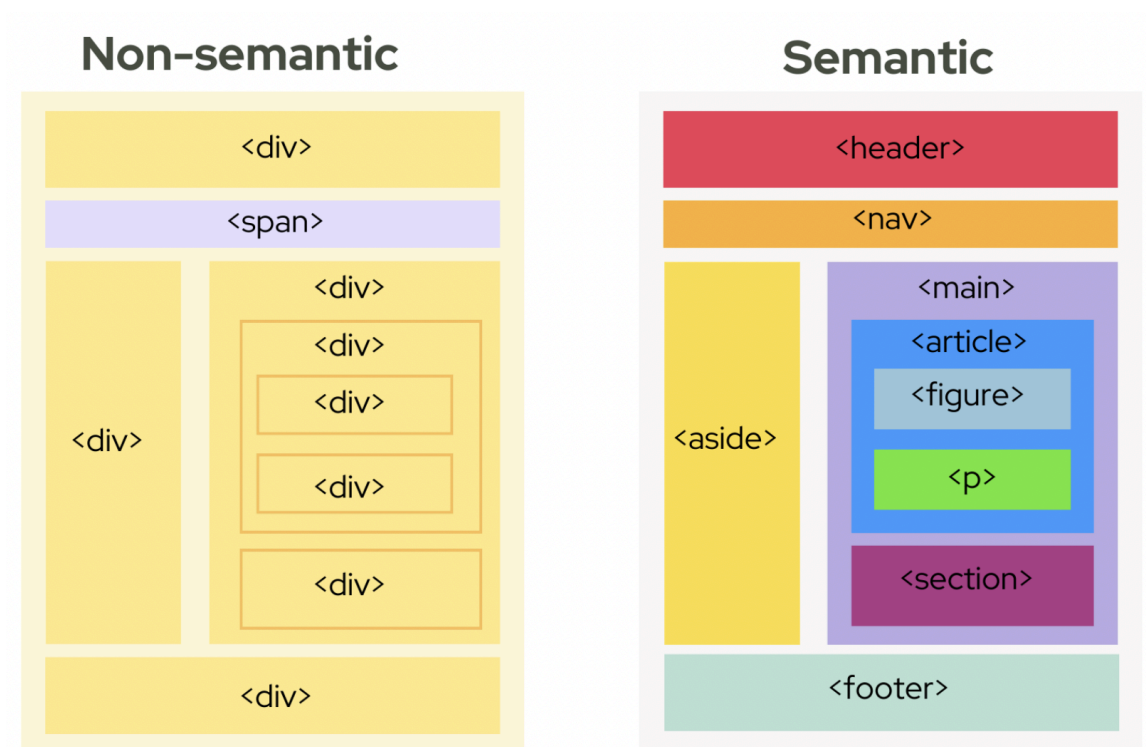
Screen readers rely on the semantic meaning of HTML tags to determine how content should be presented to users with visual impairments. Using correct HTML tags (e.g., `<header>`, `<nav>`, `<article>`) ensures that your page has a clear structure and can be easily understood by assistive devices. Take a look at the example below:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Accessible Page</title>
  </head>
  <body>
    <header>
      <h1>Welcome to Our Website</h1>
      <nav>
        <ul>
          <li><a href="#home">Home</a></li>
          <li><a href="#about">About</a></li>
          <li><a href="#contact">Contact</a></li>
        </ul>
      </nav>
    </header>

    <main>
      <article>
        <h2>Our Mission</h2>
        <p>We aim to make the web more accessible for everyone.</p>
      </article>
    </main>

    <footer>
      <p>(c) 2024 Accessible Web Inc.</p>
    </footer>
  </body>
</html>
```

In this example, we use semantic tags like `<header>`, `<nav>`, `<main>`, and `<footer>` to organise content meaningfully.



*Semantic HTML (Pavlik, 2022)*

Let's explore the advantages of using semantic HTML (right) compared to non-semantic (left) in the image just above:

Improved accessibility:

- **Screen readers:** Semantic HTML elements have specific meanings that screen readers can understand, making content accessible to users with visual impairments.
- **Keyboard navigation:** Semantic elements provide clear structure for keyboard navigation, allowing users to easily move through content without relying on visual cues.

Enhanced SEO:

- **Search engine ranking:** Search engines can better understand the content and structure of a page when it uses semantic HTML, which can improve search engine rankings.
- **Keyword relevance:** Semantic elements help search engines associate keywords with specific content areas, further improving search engine visibility.

Better maintainability and readability:

- **Clear structure:** Semantic HTML creates a more logical and understandable structure for the code, making it easier to maintain and modify.
- **Faster development:** Using semantic elements can speed up development time by providing a clear framework for building web pages.

Improved user experience:

- **Faster loading times:** Semantic HTML can lead to faster page load times, as browsers can more easily understand and render the content.
- **Better mobile experience:** Semantic HTML adapts well to different screen sizes and devices, providing a consistent user experience across platforms.

The semantic HTML approach (right) creates a well-structured, accessible, and maintainable website that not only delights users but also boosts search engine rankings.

Beyond semantic HTML, ARIA roles and attributes let us fine-tune accessibility, ensuring that every user, regardless of ability, can fully interact with and experience the website.

## ARIA roles and attributes

Accessible Rich Internet Applications (ARIA) roles and attributes help enhance accessibility for elements that might not be inherently accessible, such as interactive elements or custom widgets. ARIA provides additional information to assistive technologies to ensure that these elements can be correctly interpreted.

Some interactive elements, like dropdowns or sliders, may not be recognised by screen readers without ARIA roles and attributes. By adding ARIA, you inform assistive technologies how to present the content to users with disabilities. Take the example code just below:

```
<button aria-label="Close" onclick="closeWindow()">X</button>

<div role="navigation" aria-label="Main Navigation">
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#services">Services</a></li>
  </ul>
</div>

<article aria-labelledby="articleTitle">
  <h2 id="articleTitle">Article Heading</h2>
  <p>This is an article content.</p>
</article>
```

In the example above:

- `aria-label` is used to provide a description for the "Close" button.
- `role="navigation"` is used to define the navigation region for screen readers.
- `aria-labelledby` associates the article with its heading for a clearer structure.

Just as ARIA roles enhance navigation and structure, accessible forms – through clear labelling, required field indicators, and keyboard-friendly navigation – ensure that every user can effortlessly understand, complete, and submit essential information

## Accessible forms and inputs

Forms are one of the most essential parts of websites, but they can pose significant challenges for users with disabilities if not properly designed. Accessible forms ensure that all users, including those with visual or cognitive impairments, can fill out and submit forms with ease.

Accessible forms allow users to understand what information is required, how to navigate between fields, and how to interact with form elements (like checkboxes and radio buttons).

Proper labelling, error messages, and keyboard navigation make forms easier for all users. To illustrate how accessible forms can be implemented, here's an example of a simple form with proper labelling and keyboard navigation (in forms, this is enabled by default, allowing users to move between form fields using the **Tab** key):

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required aria-required="true">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required aria-required="true">

  <label for="message">Message:</label>
  <textarea id="message" name="message" required aria-required="true"></textarea>

  <button type="submit">Submit</button>
</form>
```

In this form example above:

- The `label` elements are properly linked to their respective input fields using the `for` attribute.
- `aria-required="true"` informs screen readers that the fields are required.

# CSS for visual accessibility

CSS plays a key role in making websites visually accessible. This includes ensuring proper contrast for readability, providing clear focus styles for keyboard navigation, and creating layouts that are easy to navigate.

Users with low vision or colour blindness need sufficient contrast between text and background. Additionally, users who navigate via keyboard rely on visible focus styles to see where they are on the page:

```
body {  
  font-size: 16px; /* Ensure readable font size */  
  color: #333; /* Dark text for high contrast */  
  background-color: #fff; /* Light background for high contrast */  
}  
  
button:focus {  
  outline: 2px solid #005fcc; /* Clear focus state for keyboard navigation */  
}  
  
input:focus, textarea:focus {  
  border: 2px solid #005fcc; /* Clear border color when input is focused */  
}  
  
a {  
  color: #005fcc; /* High-contrast link color */  
}  
  
a:focus {  
  background-color: rgba(0, 95, 204, 0.1); /* Highlight links on focus */  
}
```

In this example code above:

- We ensure a high contrast between text and background.
- Focus states are clearly visible on buttons and form inputs to assist users navigating via keyboard.
- Links are styled for clarity and accessibility.

## Colour contrast

Colour contrast refers to the difference in brightness between text and its background, which is crucial for users with visual impairments. Sufficient contrast makes text easier to read, especially for people with limited vision or difficulty distinguishing colours.

### Tips to improving colour contrast:

**Text and background:** Ensure a high contrast between text and background for readability.

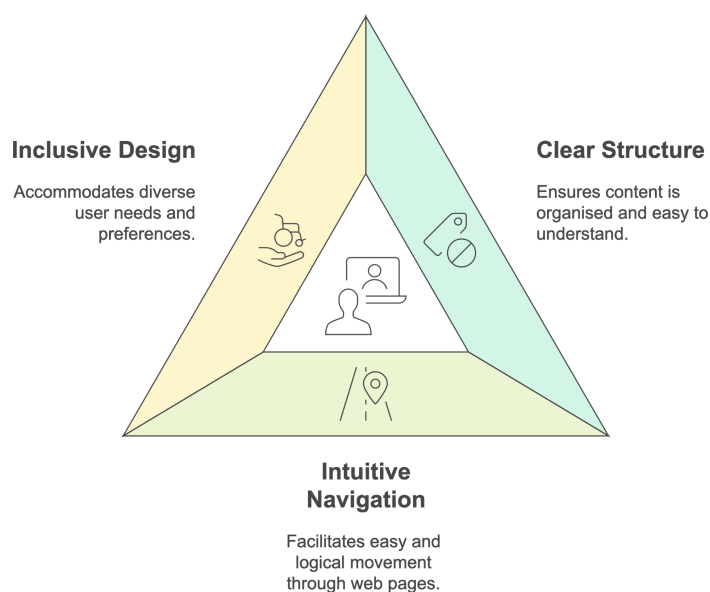
```
body {  
  color: #333; /* Dark text */  
  background-color: #fff; /* Light background */  
}
```

**Focus states and buttons:** Make focus states and buttons clear for keyboard navigation.

```
button:focus {  
  outline: 2px solid #005fcc; /* High contrast focus */  
}
```

**Avoid colour alone:** Do not use colour alone to convey meaning, add text or icons.

```
.required:before {  
  content: "*"; /* Red asterisk for required fields */  
  color: red;  
}
```



Accessibility goes beyond guidelines, aiming to create an equitable online experience that supports users with diverse needs, from visual, auditory, cognitive, or motor impairments to those accessing content non-visually. By prioritising clear structure, intuitive navigation, and inclusive design, we can make the web more accessible, where all users can fully access, interact with, and engage.



## Practical task

Follow these steps:

Create a web page (e.g., personal portfolio, contact page, or product/service landing page) using semantic HTML, ARIA roles, and accessible forms based on WCAG principles.

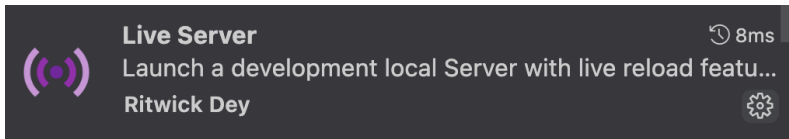
### Guidelines:

- **Semantic structure:** Use meaningful HTML tags (e.g., `<header>`, `<section>`, `<article>`) to ensure the content is logically organised and easy to navigate.
- **ARIA roles:** Integrate ARIA roles thoughtfully to improve context and navigation for users relying on assistive technologies.
- **Images:** Ensure all images have descriptive `alt` attributes to convey their purpose. Decorative images should have empty `alt` attributes (`alt=""`) to be ignored by screen readers.
- **Accessible forms:** Design a basic form with clear labels, `aria-` attributes for improved usability, and proper keyboard accessibility.

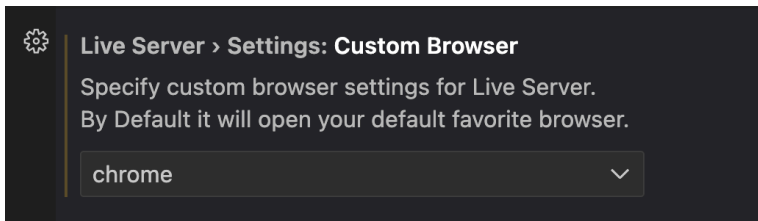
Ensure your page passes basic accessibility checks with the use of [Lighthouse](#). Before you can open your page in **Chrome**, you will need to install the **Live Server** extension in **VS Code**. Once it's running in Chrome, you can apply it for accessibility testing.

### To enable Live Server:

- Open **VS Code Settings**.
  - Go to the Extensions view (**Ctrl + Shift + X** or **Cmd + Shift + X**).
- Search for **Live Server** and ensure it's configured for Chrome as your default browser. You can adjust this setting by clicking on the 'cog' icon and navigating to 'Extension Settings'.



- Scroll down to the 'Custom Browser' setting and select in the dropdown: Chrome, like this:



- Great. Now start the Live Server by right-clicking your HTML file and choosing **"Open with Live Server"**.
- Your web page should open in Chrome. You can use Lighthouse to analyse the page's accessibility score.

### Opening Developer Tools:

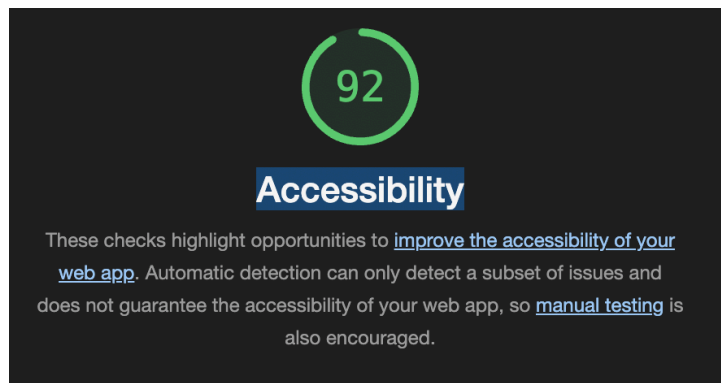
- Use the following keyboard shortcuts:
  - **Mac:**
    - Chrome/Edge/Safari: `Cmd + Option + I`
  - **Windows:**
    - Chrome/Edge: `Ctrl + Shift + I`

### Running Lighthouse:

- After opening your page in Chrome, click on the **Lighthouse** tab in Developer Tools toolbar, and select **Analyze page state** to assess the accessibility score.

### Your submission:

- **Take a screenshot of your accessibility score** before making any improvements.
- Make adjustments to your page as needed to improve accessibility based on Lighthouse feedback.
- **Take another screenshot** after making improvements to show the updated accessibility score.
- **Compile both screenshots into a PDF** and submit the document as part of your final project submission. Please label your file: **'accessibility.pdf'**.



Please ensure that you place the files for submission inside the task folder and click "Request Review" on the dashboard.

**Important:** Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



## Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.

---

## Reference list

Pavlik, V. (2022). Semantic HTML: What it is and how to use it Correctly. Semrush Blog. <https://www.semrush.com/blog/semantic-html5-guide/>